Name: Ivan Garcia **Date:** 7/28/26 **ID:** eaa108

Midterm 2

EE 3223: C++ and Data Structures — UTSA Summer 2026

Window: Thu Jul 9, 8 PM – Sat Jul 11, 8 PM Central

Covers: Lectures L8–L13 (classes, ADTs, rule of three, linked lists, stacks, queues). Parts I (L1–L7) fundamentals may appear as prerequisites but are not the focus.

Submission: Upload a single PDF with your answers for Parts A–C, plus your `.cpp` file for Part D, to Canvas before the window closes.

Academic integrity: You may use your notes, the lecture slides, and the lecture code files. You may not receive help from any person or use AI tools.

Part A — Multiple Choice (20 pts)

Circle the letter of the best answer. (4 pts each)

1. Which of the following is **not** part of the rule of three?

- ○ (a) Destructor
- ○ (b) Copy constructor
- ○ (c) Assignment operator
- ◉ (d) Default constructor

---

2. A class owns a heap array and defines a destructor but no copy constructor. When the class object is passed by value to a function, what happens?

- ○ (a) The copy constructor is not needed — destructor handles everything
- ○ (b) The compiler synthesizes a correct deep-copy constructor automatically
- ◉ (c) A shallow copy is made; both the original and the copy point to the same array, causing a double-free when both destructors run
- ○ (d) A compile error is generated

---

3. In a singly linked list with only a `head_` pointer (no tail pointer), appending to the back requires:

- ○ (a) O(1) time
- ○ (b) O(log N) time
- ◉ (c) O(N) time
- ○ (d) $O(N^2)$ time

---

4. A Stack backed by a linked list performs `push` and `pop` in:

- ○ (a) O(N) push, O(1) pop
- ○ (b) O(1) push, O(N) pop
- ○ (c) O(N) both
- ◉ (d) O(1) both

---

5. What does the self-assignment guard `if (this == &other) return *this;` in the assignment operator prevent?

- ○ (a) A stack overflow from recursive calls
- ◉ (b) Deleting the source data before copying it when a variable is assigned to itself
- ○ (c) Modifying a const object
- ○ (d) Infinite recursion in the copy constructor

Part B — Code Trace (30 pts)

Write the **exact output** each program produces. (10 pts each)

6.

```
#include <iostream>
using namespace std;
class Box {
    int* val_;
public:
    Box(int v) : val_(new int(v)) { cout << "ctor " << v << "\n"; }
    ~Box() { cout << "dtor " << *val_ << "\n"; delete val_; }
};
```

```
int main() {
    Box a(1);
    Box b(2);
    cout << "middle\n";
    return 0;
}
```

Output:

```
ctor 1
ctor 2
```

7.

```
#include <iostream>
using namespace std;
struct Node { int val; Node* next; };
int main() {
    Node* head = nullptr;
    for (int v : {10, 20, 30}) {
        head = new Node{v, head};
    }
    for (Node* p = head; p; p = p->next) { cout << p->val << " "; }
    cout << "\n";
    return 0;
}
```

Output:

```
30 20 10
```

8.

```
#include <iostream>
using namespace std;
struct Node { int val; Node* next; };
class Stack {
    Node* top_ = nullptr;
public:
    void push(int v) { top_ = new Node{v, top_}; }
    void pop()  { Node* t = top_->next; delete top_; top_ = t; }
    int  top()  const { return top_->val; }
};
int main() {
    Stack s;
    s.push(5); s.push(10); s.push(15);
    s.pop();
    cout << s.top() << "\n";
    s.push(99);
    cout << s.top() << "\n";
    return 0;
}
```

Output:

```
10
99
```

Part C — Short Answer (20 pts)

Answer in 2–4 sentences. (5 pts each)

9. What happens if a class owns a heap array but has no destructor? What happens if it has a destructor but no copy constructor?

10. What does the assignment operator need to do differently from the copy constructor? Why is the self-assignment check necessary?

11. Explain why a Queue needs two pointers (`head_` and `tail_`) while a Stack only needs one (`top_`).

12. A Queue contains {10, 20, 30} (10 at head). `dequeue()` is called, then `enqueue(99)`. What does `front()` return? Draw or describe the queue state.

Part D — Programming (30 pts)

File: `partD.cpp`

The file must compile with: `g++ -std=c++17 -Wall -Wextra -o partD partD.cpp`

Implement a `Bag` class that stores integers in a singly linked list. Duplicates are allowed. The class must provide:

- Default constructor.
- Destructor (no memory leaks).
- `void add(int v)` — insert at front; O(1).
- `bool contains(int v) const` — linear search; returns true if v is present.
- `int size() const` — number of elements.
- `void print() const` — print all values front to back, space-separated, followed by newline.

In `main()`, add {10, 20, 30, 10}, print, print size, and test `contains(20)` and `contains(99)` using `boolalpha`.

Expected output:

```
10 30 20 10
size: 4
contains 20: true
contains 99: false
```

Starter code:

```
#include <iostream>
using namespace std;
```

```
struct Node { int val; Node* next; };

class Bag {
    Node* head_;
    int   sz_;
public:
    Bag() : head_(nullptr), sz_(0) {}
    ~Bag();
    void add(int v);
    bool contains(int v) const;
    int  size() const { return sz_; }
    void print() const;
};

int main() {
    Bag b;
    b.add(10); b.add(20); b.add(30); b.add(10);
    b.print();
    cout << "size: " << b.size() << "\n";
    cout << boolalpha;
    cout << "contains 20: " << b.contains(20) << "\n";
    cout << "contains 99: " << b.contains(99) << "\n";
    return 0;
}
```

Part C — Short Answer — Answer Sheet

Type your answers to questions 9–12 below. (2–4 sentences each)

9. What happens if a class owns a heap array but has no destructor? What happens if it has a destructor but no copy constructor?

had text limit

10. What does the assignment operator need to do differently from the copy constructor? Why is the self-assignment check necessary?

had text limit

11. Explain why a Queue needs two pointers (head_ and tail_) while a Stack only needs one (top_).

had text limit

12. A Queue contains {10, 20, 30} (10 at head). dequeue() is called, then enqueue(99). What does front() return? Draw or describe the queue state.

had text limit